
ENGINEERING IMPLEMENTATION & HANDOVER GUIDE

Angular Unit Testing with Vitest, Allure, Quality Gates, GitHub Actions and IIS

A step-by-step guide to understanding, running, verifying and maintaining the Student CRUD testing and local deployment solution.

Project: Angular Student Testing

Repository: github.com/muhammad-sajjad532/student-crud

Document version: 1.0 | **Date:** 22 September 2026

Audience: Developers, reviewers, team leads and CI/CD maintainers

Contents

1. Purpose and current status

2. Solution architecture

3. Application overview

4. Prerequisites and first setup

5. Vitest and Angular configuration

6. Unit-testing concepts

7. Test-case catalogue

8. Coverage configuration

9. Allure reporting

10. Quality gates

11. Local verification procedure

12. GitHub Actions explained

13. Reading a CI result

14. Local IIS deployment

15. IIS troubleshooting

16. Acceptance-criteria mapping

17. Team-lead demonstration

18. Maintenance and next steps

19. Glossary and command reference

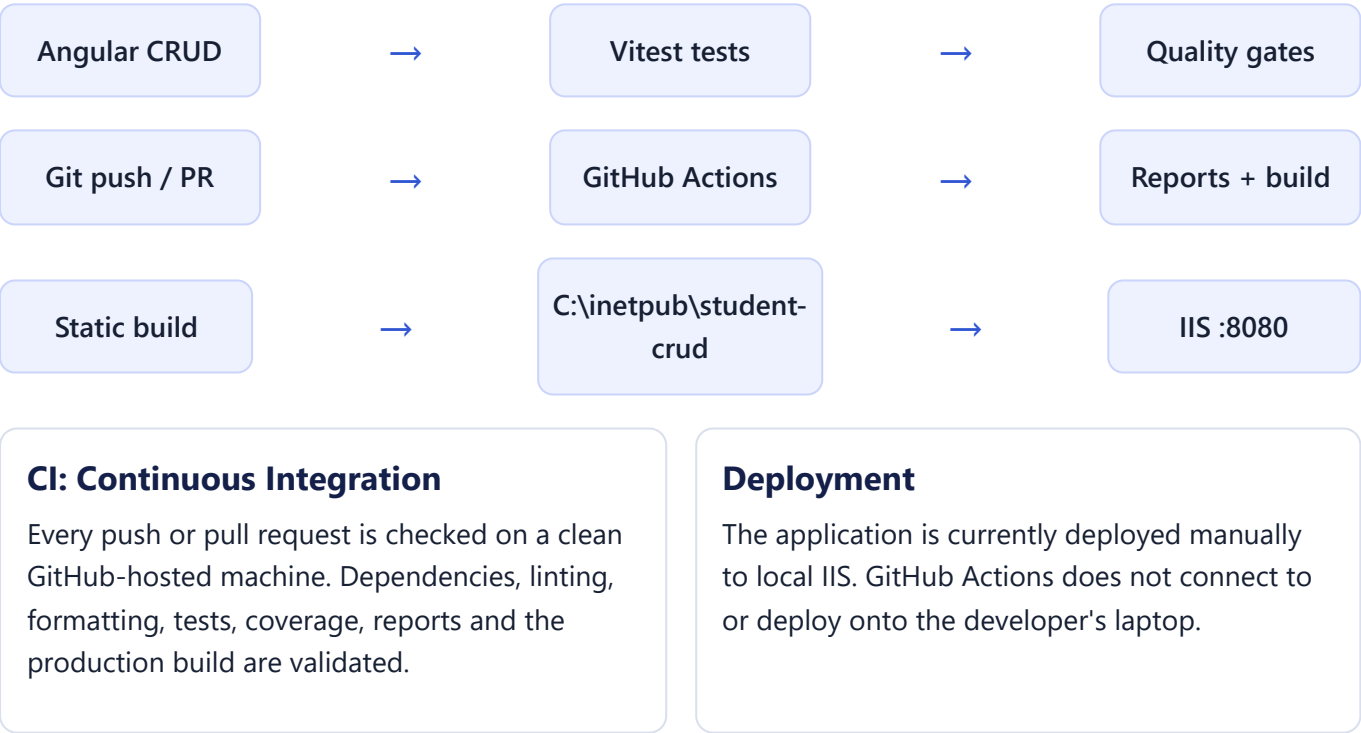
1. Purpose and current status

This project is a small standalone Angular Student CRUD application created to demonstrate a complete and repeatable unit-testing workflow. It uses Vitest to execute tests, V8 to measure code coverage, Allure Report 3 to visualize results, ESLint and Prettier for maintainability checks, GitHub Actions for continuous integration, and IIS for a local Windows deployment.

Verified baseline: 23 of 23 tests pass across two spec files. Statements, branches, functions and lines are all at 100%. Lint, formatting, result gates, Allure generation and the production build pass. The static application is available locally through IIS at `http://127.0.0.1:8080`.

The student list is intentionally stored in browser memory. Refreshing the page resets it to Ali and Sara. There is no backend API or database in this learning project.

2. Solution architecture



3. Application overview

The application exposes one standalone `StudentsComponent`. It contains a signal-based in-memory list and four public operations.

Operation	Responsibility	Important behavior
saveStudent()	Add or update a student.	Trims whitespace and rejects an empty name.
editStudent(student)	Load a student into the form.	Stores the selected ID and clears a previous error.
deleteStudent(id)	Remove a student.	Resets the form when the edited student is deleted.
cancelEdit()	Return the form to add mode.	Clears the name, selected ID and error.

The template provides an accessible form, add/save/cancel controls, a student table, edit/delete buttons, a validation alert and an empty state.

4. Prerequisites and first setup

Requirement	Project baseline	Check
Node.js	24.18.0; package accepts 24.15–24.x	<code>node --version</code>
npm	11.16.0	<code>npm --version</code>
Git	Any supported current release	<code>git --version</code>
Windows IIS	Required only for local IIS deployment	<code>Get-Service W3SVC</code>

Clone and initialize the project:

```
git clone https://github.com/muhammad-sajjad532/student-crud.git
cd student-crud
npm ci
npm start
```

Open `http://localhost:4200` for development. Stop the development server with `Ctrl+C`.

Windows note: Stop active development or test-watch processes before running `npm ci`. A running process can lock `esbuild.exe` and cause an `EPERM` installation error.

5. Vitest and Angular configuration

5.1 Angular test builder

`angular.json` uses `@angular/build:unit-test`. Angular compiles TypeScript and templates, initializes its testing environment and loads `vitest.config.ts` through `runnerConfig`.

```
"test": {
  "builder": "@angular/build:unit-test",
  "options": {
    "runnerConfig": "vitest.config.ts",
    "isolate": true
  }
}
```

Use `ng test` through the npm scripts. Running raw `npx vitest` bypasses Angular's builder integration.

5.2 Vitest safeguards

Setting	Reason
<code>allowOnly: false</code>	Rejects an accidentally committed <code>it.only</code> .
<code>passWithNoTests: false</code>	An empty suite cannot pass silently.
<code>clearMocks: true</code>	Clears mock history between tests.
<code>restoreMocks: true</code>	Restores spied functions after tests.
<code>retry: 0</code>	Does not hide an unstable failure with retries.
<code>tree reporter</code>	Shows suites and individual test names in the terminal.
<code>json reporter</code>	Creates machine-readable results for the quality gate.
<code>allure-vitest reporter</code>	Creates raw Allure result files.

6. Unit-testing concepts

6.1 Suite, case, setup and assertion

```
describe('StudentsComponent', () => {
  beforeEach(() => { /* create a fresh component */ });

  it('should add a student', () => {
    component.name = 'Ahmed'; // Arrange
    component.saveStudent(); // Act
    expect(component.students()) // Assert
      .toHaveLength(3);
  });
});
```

- **describe:** a suite that groups related behavior.
- **it:** one independently verifiable behavior.

- **beforeEach:** fresh setup before every test.
- **expect:** an assertion that decides pass or fail.

6.2 Angular TestBed and fixture

TestBed creates the Angular testing environment. `ComponentFixture` gives access to both the component instance and its rendered DOM. `fixture.whenStable()` waits for Angular bindings and events to settle.

6.3 Logic tests and DOM tests

Logic tests call component methods and inspect state. DOM tests type into an input, dispatch an event, click a button and inspect rendered HTML. The DOM is provided by `jsdom`; this is still a unit/component suite, not a full browser end-to-end test.

6.4 Independence and determinism

Every test creates a new component, uses fixed input data, avoids network calls and does not depend on execution order. This makes local and CI results repeatable.

6.5 Mocking

The current application has no HTTP API, database or external service, so no external dependency requires a mock. When a service is added, use Angular HTTP testing utilities, a fake provider or `vi.fn()`. Do not add a mock when no dependency exists.

7. Test-case catalogue

The suite contains 21 Student component tests and 2 application-shell tests.

#	Test behavior	Type	What it proves
1	Create StudentsComponent	Smoke	Angular can compile and instantiate the component.
2	Display initial students	DOM/read	Ali and Sara render as two table rows.
3	Add a student	Positive	Valid input creates ID 3 and clears the form.
4	Reject an empty name	Negative	No record is added and validation text is set.
5	Reject spaces-only name	Negative	Validation occurs after trimming.
6	Remove extra spaces	Boundary	Stored value is normalized.
7	Fill form for editing	Positive	Name and editing ID are loaded correctly.
8	Update selected student only	Positive	The target changes while other records remain intact.
9	Reject an empty edit	Negative	Original value and edit mode are preserved.
10	Cancel editing	State	Unsaved changes are discarded.
11	Delete selected student	Positive	The matching record is removed.
12	Delete currently edited student	Edge	The form safely resets.
13	Create a new ID after deletion	Edge	Deleted IDs are not accidentally reused.
14	Delete unknown ID	Negative	The existing list remains unchanged.
15	Delete another student while editing	Edge	The current edit draft is preserved.
16	Clear validation error on edit	State	A valid edit action clears stale feedback.
17	Add through HTML form	DOM	ngModel, submit and rendering work together.
18	Edit through HTML controls	DOM	Edit button, input and save flow work together.
19	Show and clear accessible error	DOM/accessibility	Alert and ARIA attributes reflect validation state.
20	Cancel through HTML control	DOM	UI returns to add mode without data changes.
21	Show empty state	DOM/edge	Deleting all students displays the expected message.
22	Create root App	Smoke	The application shell can be instantiated.
23	Render application title	DOM	The shell renders the Students feature.

8. Coverage configuration

Coverage uses the V8 provider. Application TypeScript and Angular HTML templates under `src/app` are included. Specs, interface-only models and framework wiring are excluded.

```
"coverageThresholds": {
  "perFile": true,
  "statements": 100,
  "branches": 100,
  "functions": 100,
  "lines": 100
}
```

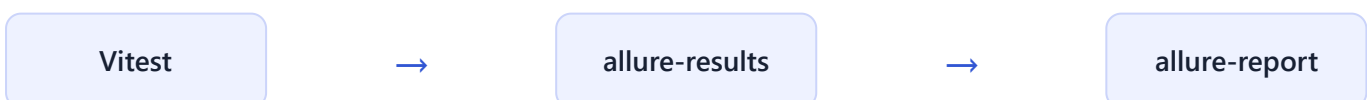
Metric	Meaning	Current result
Statements	Executable instructions reached	115/115 — 100%
Branches	Conditional alternatives reached	38/38 — 100%
Functions	Functions and handlers invoked	16/16 — 100%
Lines	Executable source lines reached	72/72 — 100%

`perFile: true` prevents a well-tested file from hiding another file with poor coverage. Coverage proves execution, not complete correctness; meaningful assertions remain essential.

```
npm run test:coverage
Start-Process .\coverage\angular-testing\index.html
```

9. Allure reporting

Vitest decides whether tests pass. Allure presents those results as a browsable report with suites, features, duration, environment and failure details.



```
npm test
npm run allure:generate
npm run allure:open
```

The tests assign the epic *Angular application* and the features *Student management* or *Application shell*. Environment metadata records Node version, platform and whether execution occurred locally or in CI.

Difference: Coverage answers “which code ran?” Allure answers “which test ran, what happened, and how long did it take?” One does not replace the other.

10. Quality gates

The main local validation command is:

```
npm run quality-check
```

Its commands are connected with &&. A failure stops the normal sequence.



Problem	Gate that fails
Incorrect expected value	Vitest assertion
<code>it.only</code>	ESLint and Vitest
<code>it.skip</code> or <code>it.todo</code>	ESLint or result-check script
Coverage below 100%	Coverage threshold
Missing/incomplete Allure result	<code>check-test-results.mjs</code>
Lint warning	ESLint with <code>--max-warnings=0</code>
Formatting difference	Prettier check
Template or compile error	Angular production build

The custom result script reads `test-results/vitest.json` and `allure-results`. It requires every discovered test to pass, rejects zero or incomplete tests and verifies a timed, named Allure result for every test.

11. Local verification procedure

1. Open a terminal in the repository root containing `package.json`.
2. Stop any active `ng serve` or `test-watch` session.
3. Install exact dependencies with `npm ci`.
4. Run the complete pipeline with `npm run quality-check`.
5. Confirm 23 tests, four 100% metrics, the quality-gate message and successful build.

6. Open reports with the commands below.

```
npm run allure:open  
Start-Process .\coverage\angular-testing\index.html
```

Useful focused commands:

Command	Purpose
npm test	Run all tests once.
npm run test:watch	Rerun tests as source files change.
npm run test:coverage	Run tests and enforce coverage thresholds.
npm run test:ci	Coverage plus JSON/Allure completeness gates.
npm run lint	Check TypeScript, templates and test conventions.
npm run format:check	Check formatting without editing files.
npm run build	Create the production SSR/browser output.

12. GitHub Actions explained

The workflow is stored at `.github/workflows/quality.yml`. GitHub reads this file after it is pushed to the repository.

12.1 Triggers

```
on:
  push:
  pull_request:
  workflow_dispatch:
```

The workflow runs automatically for pushes and pull requests. `workflow_dispatch` also adds a manual “Run workflow” button.

12.2 Runner and permissions

`runs-on: ubuntu-latest` creates a fresh temporary Linux machine. `contents: read` gives the job only the repository read permission it needs. The runner does not use local IIS, local `node_modules` or the developer's VS Code session.

12.3 Step-by-step execution

Step	Action	Reason
Checkout	<code>actions/checkout@v4</code>	Copies repository content onto the runner.
Node setup	<code>actions/setup-node@v4</code>	Installs Node 24.18.0 and enables npm caching.
Dependencies	<code>npm ci</code>	Uses the lockfile for reproducible installation.
Lint	<code>npm run lint</code>	Rejects code and test-rule violations.
Formatting	<code>npm run format:check</code>	Enforces consistent source formatting.
Test gates	<code>npm run test:ci</code>	Runs tests, coverage and result validation.
Allure	<code>npm run allure:generate</code>	Produces a readable report, including after test failures.
Build	<code>npm run build</code>	Runs only when preceding required checks succeeded.
Artifacts	<code>actions/upload-artifact@v4</code>	Preserves reports for 14 days on success or failure.

12.4 Failure behavior

A failed test returns a non-zero exit code and marks the job red. The production build is skipped because an earlier required step failed. The workflow still attempts to generate and upload available reports so the failure can be investigated.

13. Reading a CI result

1. Open the GitHub repository and select **Actions**.
2. Select **Quality checks** and open the latest run.
3. Open the **quality** job.
4. Expand each step to see the exact command and output.
5. For a failure, begin with the first red step; later skipped steps are usually consequences.
6. Download `test-reports-<run-id>-<attempt>` from the Artifacts section.

The artifact contains coverage, Vitest JSON, raw Allure results and the generated Allure report. A displayed SHA-256 value is the artifact's integrity fingerprint, not an error or deployment URL.

Branch protection recommendation: Configure the *quality* job as a required status check if pull requests must not merge until the pipeline passes.

14. Local IIS deployment

14.1 Why a separate build?

The default production build includes an Angular Node SSR server. IIS is being used here as a static file server, so the `iis` configuration sets `outputMode` to `static`.

```
npm run build:iis
```

The deployable files are generated at:

```
dist\angular-testing-iis\browser
```

14.2 Enable IIS

Run in Administrator PowerShell:

```
Enable-WindowsOptionalFeature -Online `
  -FeatureName IIS-WebServerRole,IIS-WebServer,IIS-CommonHttpFeatures,IIS-
  StaticContent,IIS-DefaultDocument,IIS-ManagementConsole `
  -All -NoRestart
```

Restart Windows only if the result says `RestartNeeded: True`.

14.3 Copy the website

```
New-Item -ItemType Directory -Path "C:\inetpub\student-crud" -Force

Copy-Item `
  -Path ".\dist\angular-testing-iis\browser\*" `
  -Destination "C:\inetpub\student-crud" `
  -Recurse -Force
```

14.4 Create the IIS application pool and site

```
& "$env:windir\System32\inetsrv\appcmd.exe" add apppool /name:"StudentCrud"

& "$env:windir\System32\inetsrv\appcmd.exe" set apppool `
  /apppool.name:"StudentCrud" /managedRuntimeVersion:""

& "$env:windir\System32\inetsrv\appcmd.exe" add site `
  /name:"StudentCrud" `
  /bindings:"http/127.0.0.1:8080:" `
  /physicalPath:"C:\inetpub\student-crud"

& "$env:windir\System32\inetsrv\appcmd.exe" set app "StudentCrud/" `
  /applicationPool:"StudentCrud"
```

14.5 Grant least-required access

```
icacls "C:\inetpub\student-crud" `
  /grant "IIS AppPool\StudentCrud:(OI)(CI)(RX)" /T
```

This grants read and execute access to the dedicated application-pool identity. It does not grant write access or broad Everyone permissions.

14.6 Verify deployment

```
& "$env:windir\System32\inetsrv\appcmd.exe" list site "StudentCrud"
curl.exe -i http://127.0.0.1:8080/
```

Expected status is state:Started and HTTP/1.1 200 OK. Open <http://127.0.0.1:8080> in the browser. VS Code and Node can be closed; IIS and Windows must remain running.

15. IIS troubleshooting

Symptom	Diagnosis	Resolution
applicationHost.config is not well-formed XML	IIS master configuration is corrupted.	Validate and restore a known-good configuration-history backup before editing.
Site or pool state is Unknown	WAS has not loaded the current configuration.	Stop W3SVC/WAS, then start WAS followed by W3SVC.
HTTP 401 despite anonymous authentication	Authentication module absent, identity wrong, or folder unreadable.	Verify modules, use application-pool identity and grant RX permission.
list modules returns nothing	IIS configuration lacks standard module registrations.	Back up IIS configuration, disable/re-enable the IIS role, then recreate the site.
Connection refused on port 8080	No HTTP listener; pool may be Unknown.	Restart WAS/W3SVC and confirm netstat -ano findstr ":8080".
Page loads but refresh loses students	Expected application design.	Add a backend/database later if persistence is required.

Safe configuration recovery

Before restoring or repairing IIS, create a backup. Validate candidate XML before replacement. IIS configuration changes affect every IIS site on the machine.

```
& "$env:windir\System32\inetsrv\appcmd.exe" list backup

$backupPath =
"$env:SystemDrive\inetpub\history\CFGHISTORY_0000000001\applicationHost.config"
$backupXml = New-Object System.Xml.XmlDocument
$backupXml.Load($backupPath)
```

Reload IIS runtime

```
Stop-Service W3SVC -ErrorAction Stop
Stop-Service WAS -Force -ErrorAction Stop
Start-Service WAS -ErrorAction Stop
Start-Service W3SVC -ErrorAction Stop
```

16. Acceptance-criteria mapping

Requirement	Implementation evidence	Status
Vitest runs against Angular	Angular unit-test builder and Vitest runner config	Complete
Critical unit tests	23 positive, negative, edge and DOM cases	Complete
Watch and CI/headless commands	test:watch, test, test:ci	Complete
Coverage reports and thresholds	V8 reports; 100% per-file thresholds	Complete
Allure visualization	Vitest reporter, generate/open/clean scripts	Complete
Failing or skipped tests rejected	Vitest, ESLint and JSON result gate	Complete
Lint and formatting	ESLint and Prettier commands in quality pipeline	Complete
Build requires successful checks	Command chain and ordered GitHub job	Complete
CI execution and artifacts	GitHub Actions workflow; 14-day reports	Complete
Local deployment	Static Angular build hosted by IIS	Complete
Automatic deployment to local IIS	No self-hosted runner configured	Not in current scope
Developer documentation	README, workshop, recipes, validation, IIS and this guide	Complete

17. Team-lead demonstration

1. Open `http://127.0.0.1:8080` and demonstrate add, edit and delete.
2. Explain one Arrange–Act–Assert logic test and one DOM test.
3. Show `runnerConfig` and the four 100% thresholds in `angular.json`.
4. Show the three reporters and safety settings in `vitest.config.ts`.
5. Run `npm run quality-check` and point out 23 tests and coverage.
6. Run `npm run allure:open` and show suites, features and duration.
7. Open the coverage HTML and show file-level coverage.
8. Open GitHub Actions and explain every workflow step.
9. Download the workflow artifact and explain the SHA-256 fingerprint.
10. Clarify that GitHub CI is automated while local IIS publishing is currently manual.

Suggested summary: "Vitest verifies application behavior. V8 measures code execution. Allure visualizes test results. ESLint and Prettier enforce maintainability. GitHub Actions runs the same gates on every push and pull request. A failed test, skipped test, coverage drop or build error makes the quality job fail. The static application is deployed locally on IIS."

18. Maintenance and next steps

Routine change workflow

1. Create or update functionality.
2. Add positive, negative and edge-case tests.
3. Run `npm run quality-check`.
4. Review `git diff` and `git status`.
5. Commit and push.
6. Verify the GitHub Actions run.
7. For IIS, run `npm run build:iis` and copy the new browser files.

Recommended future work

- Add a Student service and backend persistence, then test HTTP success and failure paths with mocks.
- Configure branch protection so the quality job is required before merge.
- Upload the IIS static build as a separate deployment artifact if delivery packaging is required.
- Use a secured GitHub self-hosted runner only if automatic deployment to this Windows IIS machine becomes a formal requirement.
- Add end-to-end browser tests when navigation and backend workflows become important.

19. Glossary and command reference

Term	Plain-language meaning
Unit test	A fast check of one focused piece of application behavior.
Assertion	The expected result that determines pass or fail.
Mock	A controlled replacement for an external dependency.
Coverage	A measurement of source code executed by tests.
Quality gate	A rule that fails the pipeline when its standard is not met.
Reporter	A component that formats or stores test results.
Artifact	A downloadable output preserved from a CI run.
Runner	The temporary machine that executes a GitHub Actions job.
CI	Automated integration validation on changes.
CD	Automated delivery or deployment after validation.
SSR	HTML rendered through a server process.
Static deployment	HTML, CSS and JavaScript served without a Node application server.

Master command list

```
npm ci
npm start
npm test
npm run test:watch
npm run test:coverage
npm run test:ci
npm run allure:generate
npm run allure:open
npm run allure:clean
npm run lint
npm run format:check
npm run quality-check
npm run build
npm run build:iis
```

Key files

File	Responsibility
package.json	Dependencies and developer/CI commands.
angular.json	Build, test, coverage and IIS-static configurations.
vitest.config.ts	Vitest safeguards and reporters.
eslint.config.mjs	Source, Angular template and test lint rules.
students.component.spec.ts	Student component behavior and DOM tests.
app.spec.ts	Application shell smoke tests.
scripts/check-test-results.mjs	No-skip and Allure-completeness gate.
scripts/clean-reports.mjs	Safely removes generated result directories.
.github/workflows/quality.yml	GitHub-hosted CI pipeline.
docs/IIS-DEPLOYMENT.md	Focused local IIS instructions.

20. References

- [Angular testing documentation](#)
- [Angular code coverage](#)
- [Angular rendering modes](#)
- [Allure Report 3 documentation](#)
- [GitHub Actions documentation](#)
- [Microsoft IIS static website guide](#)